🗒️ TiDPI 06 — Ustrukturyzowany output, parser, CSV i walidacja danych

📔 Tematy do dziennika

```
[TiDPI] Kontrakt danych wyjściowych programu — wersja formatu, nagłówek, rekordy, typy pól i jednoznaczne jednostki
[TiDPI] Parser i wielowarstwowa walidacja danych — kontrola struktury, typów, zakresów, słowników wartości i unikalności rekordów
[TiDPI] Bezpieczny zapis zweryfikowanych wyników do CSV — moduł csv, separator, kodowanie, kody zakończenia i dowody odrzucenia danych
```

🎯 Cel bloku

Po tym bloku będziecie potrafili zaprojektować maszynowy format wyjścia programu, napisać parser odróżniający poprawne dane od uszkodzonych, zwalidować strukturę i znaczenie pól oraz zapisać do CSV wyłącznie rekordy, które przeszły wszystkie kontrole.

Rezultatem będzie jedno anonimowe sprawozdanie w Markdownie zawierające kontrakt danych, przypadek poprawny, kontrolowane przypadki błędne, kody zakończenia, surowe pliki wejściowe i uzasadnienie decyzji parsera.

🔁 Od logu dla człowieka do danych dla programu

W lekcji 05 automat zachowywał `stdout`, `stderr` i kody zakończenia. Surowy log jest dobrym dowodem, ale drugi program musi jeszcze wiedzieć, gdzie w nim znajduje się nazwa przypadku, rozmiar danych, numer próby i wynik kontroli poprawności.

Rozważmy komunikat:

```
Sortowanie dużej tablicy poszło całkiem nieźle: około 7 ms :)
```

Człowiek domyśli się wielu rzeczy. Parser zacznie zadawać niewygodne, lecz słuszne pytania:

- Co znaczy „duża”?
- Czy `7 ms` jest wynikiem dokładnym, czy zaokrąglonym?
- Jakiego algorytmu użyto?
- Która to próba?
- Czy wynik sortowania był poprawny?
- Czy emotikon należy do danych?

Program przeznaczony do dalszej automatycznej analizy powinien emitować **ustrukturyzowany output**, czyli tekst z ustalonym podziałem na rekordy i pola. Ten podział jest umową pomiędzy producentem danych a ich odbiorcą.

> Parser nie jest wróżbitą. Jeżeli każemy mu odgadywać znaczenie po przecinkach, nawiasach i dobrym humorze autora komunikatu, kiedyś przepowie katastrofę — i będzie to najbardziej deterministyczna część systemu.

📜 Kontrakt danych

Kontrakt danych opisuje format, znaczenie i ograniczenia informacji wymienianych między programami. Nasz kontrakt będzie miał trzy części:

```
TIDPI_RESULTS;1
case;size;trial;time_us;comparisons;correct
sorted;16;1;2;15;true
```

Pierwszy wiersz zawiera nazwę protokołu i jego wersję. Drugi jest nagłówkiem określającym kolejność kolumn. Każdy następny wiersz jest jednym rekordem wyniku.

🔢 Dlaczego format ma wersję?

Dzisiaj rekord ma sześć pól. Za miesiąc ktoś może dodać kolumnę `seed`, zmienić jednostkę czasu albo przemianować `case`. Bez numeru wersji stary parser mógłby zinterpretować nowe dane w niewłaściwy sposób.

Wersja `1` nie znaczy „format doskonały na wieki”. Znaczy: „interpretuj ten plik dokładnie według zasad wersji pierwszej”. Parser, który rozumie tylko wersję `1`, ma odrzucić wersję `99`, zamiast zgadywać.

> Numer wersji jest jak etykieta na kablu w pracowni. Można uznać ją za zbędną — aż do chwili, gdy trzeba ustalić, który z dwunastu identycznych przewodów właśnie odłączył internet całej klasie.

🧩 Schemat rekordu

Schemat określa nazwy pól, ich typy i reguły. W naszym eksperymencie przyjmujemy:

Pole	Typ po walidacji	Reguła	Znaczenie
`case`	tekst	`sorted`, `reversed` albo `random`	układ danych wejściowych
`size`	liczba całkowita	co najmniej `2`	liczba elementów
`trial`	liczba całkowita	co najmniej `1`	numer próby
`time_us`	liczba całkowita	co najmniej `0`	czas w mikrosekundach
`comparisons`	liczba całkowita	co najmniej `0`	liczba porównań
`correct`	wartość logiczna	dokładnie `true` albo `false`	zgodność z wynikiem wzorcowym

Jednostkę umieszczamy w nazwie `time_us`, a nie w każdej komórce. Dzięki temu wartość `17` jest liczbą, nie napisem `17 mikrosekund`, `17 us` ani `około 17`. Jeżeli zmienimy jednostkę, musimy zmienić również nazwę pola lub wersję kontraktu.

📄 CSV — prosta tabela tekstowa

CSV pochodzi od *Comma-Separated Values*, czyli wartości rozdzielanych przecinkami. W praktyce spotkacie również pliki rozdzielane średnikami, tabulatorami i innymi znakami. W naszym kontrakcie separatorem jest średnik `;`, ponieważ taki wariant bywa wygodniejszy w polskim Excelu, gdzie przecinek często pełni funkcję separatora dziesiętnego.

To nie znaczy, że wystarczy połączyć wartości instrukcją `';'.join(...)`. Pole tekstowe może samo zawierać separator, cudzysłów albo znak nowego wiersza. Moduł `csv` potrafi wtedy prawidłowo zastosować cudzysłowy i reguły ucieczki.

Do zapisu użyjemy:

```
writer = csv.DictWriter(
    target,
    fieldnames=EXPECTED_HEADER,
    delimiter=";",
    lineterminator="\n",
)
```

`DictWriter` pobiera słowniki i zapisuje wartości zgodnie z kolejnością `fieldnames`. Argument `newline=''` przy otwieraniu pliku zapobiega dokładaniu pustych wierszy przez warstwy obsługi tekstu w Windows. Kodowanie `utf-8-sig` dodaje znacznik BOM, który pomaga niektórym wersjom Excela rozpoznać UTF-8. Ceną są dodatkowe bajty na początku pliku; parser używa tego samego kodowania i potrafi je pominąć.

🔍 Parsowanie i walidacja to nie to samo

Parsowanie (*parsing*) zamienia tekst na strukturę, na przykład listę pól i słownik rekordu. **Walidacja** sprawdza, czy otrzymana struktura spełnia ustalone reguły.

Wiersz:

```
random;dużo;1;5;120;true
```

da się podzielić na sześć pól, więc parsowanie CSV się uda. Walidacja odrzuci jednak wartość `dużo`, ponieważ `size` ma być liczbą całkowitą nie mniejszą niż `2`.

Z kolei rekord:

```
random;32;1;5;120;false
```

jest **poprawnym rekordem opisującym nieudany test**. Parser nie powinien nazywać go uszkodzonym. Dane są prawidłowe, a pole `correct=false` przekazuje ważną informację o badanym programie.

To rozróżnienie będzie widoczne w kodach zakończenia parsera:

Kod	Znaczenie
`0`	dane są poprawne i wszystkie rekordy mają `correct=true`
`1`	dane są poprawne, CSV powstał, lecz co najmniej jeden test ma `correct=false`
`2`	wejście lub jego struktura są niepoprawne; CSV nie powstaje

✅ Warstwy walidacji

Dobry parser nie kończy kontroli na policzeniu średników. Sprawdzimy kilka warstw:

- **odczyt** — czy plik istnieje, ma obsługiwane kodowanie i daje się przeczytać;
- **protokół** — czy nazwa i wersja formatu są obsługiwane;
- **nagłówek** — czy nazwy oraz kolejność kolumn są dokładnie takie, jak w kontrakcie;
- **liczba pól** — czy każdy rekord ma sześć komórek;
- **typy** — czy liczby rzeczywiście można zamienić na `int`;
- **zakresy** — czy `size`, `trial`, czas i licznik nie mają niedozwolonych wartości;
- **słownik wartości** — czy `case` i `correct` należą do jawnie dozwolonych zbiorów;
- **unikalność** — czy identyfikator `(case, size, trial)` nie powtarza się;
- **obecność danych** — czy plik zawiera przynajmniej jeden rekord.

Niektóre reguły zależą od konkretnego eksperymentu. Gdy kontrakt obiecuje dokładnie trzy próby dla każdego przypadku, można dodać kontrolę kompletności. Nasz parser nie zakłada jeszcze konkretnej liczby rozmiarów i prób, aby można go było wykorzystać po rozszerzeniu eksperymentu.

🛑 Fail fast czy zebrać wszystkie błędy?

Fail fast oznacza przerwanie pracy po pierwszym błędzie. Jest bezpieczne i proste, ale użytkownik może poprawiać plik po jednym problemie. Nasz parser zbiera możliwie wiele niezależnych błędów, po czym odrzuca cały zestaw. Nie zapisuje częściowo poprawnego CSV.

To podejście nazywamy **all or nothing**: albo wszystkie rekordy przechodzą walidację, albo wynikowy plik nie powstaje. Dzięki temu dalszy etap nie pomyli niepełnego zbioru z kompletnymi danymi.

👻 Stary CSV jest równie podejrzany jak stary EXE

Załóżmy, że wczoraj parser utworzył poprawny `results.csv`. Dzisiaj wejście zawiera błąd, ale skrypt pozostawia w katalogu wczorajszy plik. Ktoś otwiera CSV i uznaje go za wynik bieżącego przebiegu.

Dlatego parser przed rozpoczęciem pracy wykonuje:

```
args.output.unlink(missing_ok=True)
```

Jeżeli walidacja się nie powiedzie, starego CSV już nie ma, a nowy nie zostanie utworzony. Jest to ta sama zasada, którą w lekcji 05 zastosowaliśmy do pliku EXE.

> Automatyzacja ma niezwykły talent do produkowania bardzo wiarygodnie wyglądających nieporozumień. Usunięcie starego artefaktu jest małym gestem, który potrafi uratować duży raport.

🎾 Czy potrzebujemy wyrażeń regularnych?

Wyrażenie regularne (*regular expression*, *regex*) jest wzorcem służącym do rozpoznawania tekstu. Regex przydaje się, gdy nie kontrolujemy starego formatu, na przykład:

```
Czas sortowania (indeksy): 0.123456 s
```

Możemy wtedy użyć wzorca:

```
pattern = re.compile(
    r"^Czas sortowania \((?P<mode>indeksy|wskazniki)\): "
    r"(?P<time>\d+(?:\.\d+)?) s$"
)
```

Najważniejsze elementy tego przykładu:

Fragment	Znaczenie
`r"..."`	surowy napis Pythona, w którym ukośniki są wygodniejsze do zapisu
`^` i `$`	początek i koniec wiersza — dopasowujemy całość, nie przypadkowy fragment
`\(` i `\)`	dosłowne nawiasy
`(?P<mode>...)`	nazwana grupa `mode`
`` `indeksy ``wskazniki``	wskazniki`
`\d+`	co najmniej jedna cyfra
`(?:\.\d+)?`	opcjonalna część dziesiętna

Regex nie zastępuje późniejszej konwersji typu, kontroli zakresu ani sprawdzenia duplikatów. Co więcej, drobna zmiana komunikatu przeznaczonego dla człowieka może złamać wzorzec.

Jeżeli kontrolujemy producenta i odbiorcę danych, zwykle lepiej wyemitować CSV, JSON albo inny jawny format i użyć gotowego parsera. W tej lekcji głównym narzędziem jest więc moduł `csv`; regex pozostaje rozwiązaniem dla uzasadnionych przypadków tekstu zastanego.

> Regex jest jak bardzo precyzyjna pułapka na myszy. Świetnie działa na mysz o wymiarach przewidzianych przez konstruktora. Gorzej, gdy następnego dnia przychodzi chomik (z dodatkową spacją).

💻 Program C++ emitujący ustrukturyzowany output

Program wykonuje niewielkie próby sortowania dla trzech układów i dwóch rozmiarów. Czas jest tylko przykładowym polem do przetworzenia — nie wyciągamy z niego jeszcze wniosków benchmarkowych. Rzetelną metodykę benchmarku omówimy w lekcji 07.

Argument `--fault` pozwala świadomie wygenerować niepoprawne dane albo poprawny rekord nieudanego testu. Dzięki temu sprawdzimy obie gałęzie parsera bez ręcznego poprawiania surowych dowodów.

```
#include <algorithm>
#include <chrono>
#include <cstdint>
#include <iostream>
#include <random>
#include <stdexcept>
#include <string>
#include <vector>

struct SortResult
{
    std::string test_case;
    std::size_t size;
    int trial;
    long long time_us;
    std::uint64_t comparisons;
    bool correct;
};

enum class FaultMode
{
    none,
    invalid_number,
    missing_field,
    duplicate_row,
    failed_test,
    wrong_version
};

FaultMode parse_fault_mode(int argc, char* argv[])
{
    // Tryby awarii celowo wytwarzają konkretne błędy danych. Dzięki nim sprawdzamy
    // każdą ścieżkę odrzucenia parsera bez przypadkowego uszkadzania programu.
    if (argc == 1)
    {
        return FaultMode::none;
    }

    if (argc != 3 || std::string(argv[1]) != "--fault")
    {
        throw std::invalid_argument("usage: program [--fault MODE]");
    }

    const std::string mode = argv[2];

    if (mode == "invalid-number")
    {
        return FaultMode::invalid_number;
    }
    if (mode == "missing-field")
    {
        return FaultMode::missing_field;
    }
    if (mode == "duplicate-row")
```

```
    {
        return FaultMode::duplicate_row;
    }
    if (mode == "failed-test")
    {
        return FaultMode::failed_test;
    }
    if (mode == "wrong-version")
    {
        return FaultMode::wrong_version;
    }

    throw std::invalid_argument("unknown fault mode");
}

std::vector<int> make_data(const std::string& test_case, std::size_t size, int trial)
{
    std::vector<int> values(size);

    for (std::size_t i = 0; i < size; ++i)
    {
        values[i] = static_cast<int>(i);
    }

    if (test_case == "reversed")
    {
        std::reverse(values.begin(), values.end());
    }
    else if (test_case == "random")
    {
        // Osobny generator z zapisanym ziarnem odtwarza dane danej próby niezależnie
        // od kolejności uruchomień, więc wynik można później powtórzyć.
        const std::uint32_t seed =
            20260600U + static_cast<std::uint32_t>(size * 10) +
            static_cast<std::uint32_t>(trial);
        std::mt19937 generator(seed);
        std::shuffle(values.begin(), values.end(), generator);
    }

    return values;
}

std::uint64_t bubble_sort(std::vector<int>& values)
{
    std::uint64_t comparisons = 0;

    for (std::size_t pass = 0; pass + 1 < values.size(); ++pass)
    {
        bool swapped = false;

        for (std::size_t i = 0; i + 1 < values.size() - pass; ++i)
        {
            ++comparisons;

            if (values[i] > values[i + 1])
            {
                std::swap(values[i], values[i + 1]);
                swapped = true;
            }
        }

        if (!swapped)
        {
```

```
// Brak zamiany oznacza, że dane są już uporządkowane i dalsze przebiegi
            // nie zmienią wyniku.
            break;
        }
    }

    return comparisons;
}

SortResult run_trial(const std::string& test_case, std::size_t size, int trial)
{
    std::vector<int> values = make_data(test_case, size, trial);
    std::vector<int> expected = values;
    std::sort(expected.begin(), expected.end());

    // Mierzymy wyłącznie sortowanie. Przygotowanie danych i wyniku wzorcowego
    // pozostaje poza przedziałem, aby nie zawyżać czasu badanej operacji.
    const auto start = std::chrono::steady_clock::now();
    const std::uint64_t comparisons = bubble_sort(values);
    const auto stop = std::chrono::steady_clock::now();

    const long long time_us =
        std::chrono::duration_cast<std::chrono::microseconds>(stop - start).count();

    return SortResult
    {
        test_case,
        size,
        trial,
        time_us,
        comparisons,
        values == expected
    };
}

void emit_row(const SortResult& result, FaultMode fault_mode, bool first_row)
{
    // Psujemy tylko pierwszy rekord, aby każda próba sprawdzała jedną regułę
    // walidacji i było wiadomo, która nieprawidłowość wywołała reakcję parsera.
    if (first_row && fault_mode == FaultMode::invalid_number)
    {
        std::cout
            << result.test_case << ";many;" << result.trial << ';'
            << result.time_us << ';' << result.comparisons << ';'
            << std::boolalpha << result.correct << '\n';
        return;
    }

    std::cout
        << result.test_case << ';' << result.size << ';' << result.trial << ';'
        << result.time_us << ';' << result.comparisons;

    if (!(first_row && fault_mode == FaultMode::missing_field))
    {
        std::cout << ';' << std::boolalpha << result.correct;
    }

    std::cout << '\n';
}

int main(int argc, char* argv[])
{
    FaultMode fault_mode = FaultMode::none;
```

```
    try
    {
        fault_mode = parse_fault_mode(argc, argv);
    }
    catch (const std::invalid_argument& error)
    {
        std::cerr << "ERROR: " << error.what() << '\n';
        return 2;
    }

    std::vector<SortResult> results;
    const std::vector<std::string> test_cases{ "sorted", "reversed", "random" };
    const std::vector<std::size_t> sizes{ 16, 32 };

    for (const std::string& test_case : test_cases)
    {
        for (const std::size_t size : sizes)
        {
            for (int trial = 1; trial <= 3; ++trial)
            {
                results.push_back(run_trial(test_case, size, trial));
            }
        }
    }

    if (fault_mode == FaultMode::failed_test)
    {
        // Rekord zachowuje poprawny format, ale opisuje nieudany test. Sprawdzamy w ten
        // sposób, czy parser odróżnia poprawność danych od wyniku samego testu.
        results.front().correct = false;
    }

    // Nazwa protokołu i wersja mówią parserowi, jak rozumieć pola. Bez nich nowy
    // format mógłby zostać cicho odczytany według starych, błędnych zasad.
    std::cout
        << "TIDPI_RESULTS;"
        << (fault_mode == FaultMode::wrong_version ? "99" : "1") << '\n'
        << "case;size;trial;time_us;comparisons;correct\n";

    for (std::size_t i = 0; i < results.size(); ++i)
    {
        emit_row(results[i], fault_mode, i == 0);
    }

    if (fault_mode == FaultMode::duplicate_row)
    {
        // Powtarzamy rekord o tym samym przypadku, rozmiarze i numerze próby. Te trzy
        // pola razem identyfikują pomiar, więc parser powinien wykryć duplikat.
        emit_row(results.back(), FaultMode::none, false);
    }

    // Poprawny format nie oznacza zaliczonego testu. Kod wyjścia 1 informuje
    // automatyzację, że dane są czytelne, ale co najmniej jeden test nie przeszedł.
    return fault_mode == FaultMode::failed_test ? 1 : 0;
}
```

Program przygotowuje wszystkie rekordy przed ich wypisaniem. Tryby `invalid-number`, `missing-field`, `duplicate-row` i `wrong-version` celowo łamią kontrakt. Tryb `failed-test` nie psuje formatu: ustawia `correct=false` i kończy program kodem `1`.

Parser i walidator w Pythonie

Parser najpierw usuwa stary plik wyjściowy, wczytuje rekordy modułem `csv`, a potem przeprowadza walidację. CSV zapisuje dopiero wtedy, gdy lista błędów pozostaje pusta.

```
"""Walidować wynik eksperymentu i publikować CSV tylko dla wiarygodnych danych."""

import argparse
import csv
import sys
from pathlib import Path


PROTOCOL_NAME = "TIDPI_RESULTS"
SUPPORTED_VERSION = "1"
EXPECTED_HEADER = [
    "case",
    "size",
    "trial",
    "time_us",
    "comparisons",
    "correct",
]
ALLOWED_CASES = {"sorted", "reversed", "random"}


def parse_integer(text, field_name, line_number, minimum, errors):
    """Sprawdzić jedno pole całkowite i zapisać opis wykrytego błędu.

    Args:
        text: Tekst odczytany z pola źródłowego.
        field_name: Nazwa pola używana w komunikacie diagnostycznym.
        line_number: Numer wiersza źródła liczony od jednego.
        minimum: Najmniejsza dopuszczalna wartość.
        errors: Modyfikowana lista, do której dopisywane są błędy walidacji.

    Returns:
        Odczytana liczba albo None, gdy pole nie przejdzie walidacji.
    """
    try:
        value = int(text)
    except ValueError:
        errors.append(
            f"line {line_number}: {field_name} must be an integer, got {text!r}"
        )
        return None

    if value < minimum:
        errors.append(
            f"line {line_number}: {field_name} must be >= {minimum}, got {value}"
        )
        return None

    return value


def parse_boolean(text, line_number, errors):
    """Sprawdzić wymagany przez protokół zapis wartości logicznej małymi literami.

    Args:
        text: Tekst odczytany z pola źródłowego.
        line_number: Numer wiersza źródła liczony od jednego.
        errors: Modyfikowana lista, do której dopisywane są błędy walidacji.
```

```
    Returns:
        True, False albo None, gdy pole nie przejdzie walidacji.
    """
    if text == "true":
        return True
    if text == "false":
        return False

    errors.append(
        f"line {line_number}: correct must be 'true' or 'false', got {text!r}"
    )
    return None


def validate_rows(rows):
    """Sprawdzić cały protokół bez publikowania częściowych wyników.

    Args:
        rows: Wiersze podzielone już na pola przez czytnik CSV.

    Returns:
        Para zawierająca poprawne, ujednolicone rekordy i wszystkie komunikaty.
    """
    errors = []
    records = []
    identities = set()

    if len(rows) < 2:
        return [], ["file must contain a protocol line and a header"]

    # Sprawdzamy dokładnie wersję i nagłówek, aby przesunięte lub zmienione kolumny
    # nie zostały cicho zinterpretowane według nieaktualnych zasad.
    if rows[0] != [PROTOCOL_NAME, SUPPORTED_VERSION]:
        errors.append(
            "line 1: expected "
            f"{PROTOCOL_NAME};{SUPPORTED_VERSION}, got {';'.join(rows[0])!r}"
        )

    if rows[1] != EXPECTED_HEADER:
        errors.append(
            f"line 2: expected header {';'.join(EXPECTED_HEADER)!r}, "
            f"got {';'.join(rows[1])!r}"
        )

    for line_number, row in enumerate(rows[2:], start=3):
        if len(row) != len(EXPECTED_HEADER):
            errors.append(
                f"line {line_number}: expected {len(EXPECTED_HEADER)} fields, "
                f"got {len(row)}"
            )
            continue

        error_count_before = len(errors)
        test_case, size_text, trial_text, time_text, comparisons_text, correct_text = row

        if test_case not in ALLOWED_CASES:
            errors.append(
                f"line {line_number}: unknown case {test_case!r}; "
                f"allowed: {', '.join(sorted(ALLOWED_CASES))}"
            )

        size = parse_integer(size_text, "size", line_number, 2, errors)
```

```
        trial = parse_integer(trial_text, "trial", line_number, 1, errors)
        time_us = parse_integer(time_text, "time_us", line_number, 0, errors)
        comparisons = parse_integer(
            comparisons_text,
            "comparisons",
            line_number,
            0,
            errors,
        )
        correct = parse_boolean(correct_text, line_number, errors)

        if size is not None and trial is not None:
            # Sama nazwa przypadku nie jest unikatowa. Dopiero przypadek, rozmiar i numer
            # próby razem wskazują jeden pomiar, więc ten zestaw wykrywa duplikaty.
            identity = (test_case, size, trial)
            if identity in identities:
                errors.append(
                    f"line {line_number}: duplicate record identity {identity!r}"
                )
            else:
                identities.add(identity)

        if len(errors) == error_count_before:
            records.append(
                {
                    "case": test_case,
                    "size": size,
                    "trial": trial,
                    "time_us": time_us,
                    "comparisons": comparisons,
                    "correct": "true" if correct else "false",
                }
            )

    if len(rows) == 2:
        errors.append("file contains no result records")

    return records, errors


def read_rows(input_path):
    """Odczytać z pliku UTF-8 wiersze rozdzielone średnikami.

    Args:
        input_path: Ścieżka surowego wyniku eksperymentu.

    Returns:
        Lista wierszy, z których każdy jest listą pól.

    Raises:
        OSError: Gdy pliku nie można otworzyć albo odczytać.
        UnicodeError: Gdy wejście nie jest poprawnym tekstem UTF-8.
        csv.Error: Gdy parser CSV nie potrafi odczytać struktury.
    """
    with input_path.open("r", encoding="utf-8-sig", newline="") as source:
        return list(csv.reader(source, delimiter=";"))


def write_csv(output_path, records):
    """Zastąpić plik docelowy ujednoliconymi rekordami rozdzielonymi średnikami.

    Args:
        output_path: Ścieżka docelowego pliku CSV.
```

```
        records: Zweryfikowane rekordy zgodne z EXPECTED_HEADER.

    Raises:
        OSError: Gdy nie można zapisać pliku docelowego.
    """
    with output_path.open("w", encoding="utf-8-sig", newline="") as target:
        writer = csv.DictWriter(
            target,
            fieldnames=EXPECTED_HEADER,
            delimiter=";",
            lineterminator="\n",
        )
        writer.writeheader()
        writer.writerows(records)


def main():
    """Zweryfikować plik wejściowy i zwrócić status zrozumiały dla automatyzacji."""
    parser = argparse.ArgumentParser(
        description="Validate TiDPI result output and create a clean CSV file."
    )
    parser.add_argument("input", type=Path, help="raw output from the C++ program")
    parser.add_argument("output", type=Path, help="validated CSV destination")
    args = parser.parse_args()

    # Usuwamy stary CSV przed walidacją, aby po odrzuceniu nowych danych nie pozostał
    # plik z poprzedniej próby, który wyglądałby jak aktualny wynik.
    args.output.unlink(missing_ok=True)

    # Zapisujemy CSV dopiero po sprawdzeniu wszystkich wierszy. Jeden błędny rekord
    # odrzuca cały zestaw, więc nie publikujemy mylących wyników częściowych.
    try:
        rows = read_rows(args.input)
    except (OSError, UnicodeError, csv.Error) as error:
        print(f"VALIDATION ERROR: cannot read input: {error}", file=sys.stderr)
        return 2

    records, errors = validate_rows(rows)

    if errors:
        for error in errors:
            print(f"VALIDATION ERROR: {error}", file=sys.stderr)
        print(f"REJECTED: {len(errors)} validation error(s)", file=sys.stderr)
        return 2

    try:
        write_csv(args.output, records)
    except OSError as error:
        print(f"OUTPUT ERROR: cannot write CSV: {error}", file=sys.stderr)
        return 2

    failed_tests = sum(record["correct"] == "false" for record in records)
    print(f"ACCEPTED: {len(records)} record(s), failed tests: {failed_tests}")

    return 1 if failed_tests else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

🧭 Jak czytać najważniejsze fragmenty parsera?

`EXPECTED_HEADER` i `ALLOWED_CASES` zapisują część kontraktu w jednym, łatwym do odnalezienia miejscu. Ścisłe porównanie nagłówka wykrywa również zmianę kolejności kolumn.

`parse_integer` nie tylko wykonuje `int(text)`, ale dodaje czytelny błąd z numerem wiersza i sprawdza dolną granicę. Funkcja zwraca `None`, gdy wartości nie wolno dalej traktować jak liczby.

Zbiór `identities` przechowuje trójki `(case, size, trial)`. Zbiór pozwala szybko sprawdzić, czy identyczny rekord logiczny już się pojawił.

Warunek `len(errors) == error_count_before` powoduje, że rekord trafia na listę wyników tylko wtedy, gdy żadna z jego kontroli nie dodała błędu. Nawet te poprawne rekordy nie zostaną jednak zapisane, jeżeli błąd występuje gdziekolwiek w pliku.

▶️ Uruchomienie pełnego przebiegu

Po zbudowaniu programu w Release przejdźcie do katalogu roboczego zawierającego EXE i parser. Zapiszcie surowe wyjście, jawnie wybierając UTF-8 zamiast polegać na domyślnym kodowaniu konkretnej wersji PowerShella:

```
.\L06_Ustrukturyzowany_output.exe | Set-Content -Encoding utf8 raw_output.txt
$LASTEXITCODE
```

Następnie uruchomcie parser:

```
python .\parse_results.py raw_output.txt results.csv
$LASTEXITCODE
```

Dla poprawnych danych powinniście otrzymać komunikat podobny do:

```
ACCEPTED: 18 record(s), failed tests: 0
```

Nie przepisujcie przykładowych czasów z lekcji do raportu. Plik `results.csv` ma zawierać wartości rzeczywiście wyemitowane na Waszym komputerze.

🧨 Kontrolowane błędy kontraktu

Niepoprawną liczbę wygenerujecie poleceniem:

```
.\L06_Ustrukturyzowany_output.exe --fault invalid-number | Set-Content -Encoding utf8
raw_invalid.txt
python .\parse_results.py raw_invalid.txt invalid.csv
$LASTEXITCODE
```

Analogicznie dostępne są:

```
missing-field
duplicate-row
wrong-version
failed-test
```

Dla pierwszych czterech parser ma zwrócić `2`, podać przyczynę na `stderr` i nie pozostawić CSV. Dla `failed-test` dane są poprawne: CSV ma powstać, a parser zwrócić `1`.

🔬 Weryfikacja CSV

Nie wystarczy zobaczyć ikonę Excela. Sprawdźcie plik na trzy sposoby:

- otwórzcie go w edytorze tekstowym i zobaczcie separator, nagłówek oraz surowe wartości;
- zaimportujcie go w Excelu przez **Dane → Z tekstu/CSV**, jawnie potwierdzając kodowanie UTF-8 i separator `;`;
- wczytajcie go ponownie modułem `csv` albo prostym skryptem i policzcie rekordy.

Dwukrotne kliknięcie może użyć ustawień regionalnych systemu i przypadkiem źle rozpoznać kolumny. To nie musi oznaczać uszkodzenia samego pliku — importer i format również zawierają ustawienia.

🧪 Ćwiczenia i eksperyment

Wykonujcie poziomy kolejno. Surowych plików wyjściowych nie poprawiajcie ręcznie. Wykorzystujcie kontrolowane tryby programu albo twórzcie osobne pliki testowe, jasno oznaczone jako celowo uszkodzone dane. Taki przygotowany i kontrolowany zestaw wejściowy nazywamy **fixture danych testowych**; w liczbie mnogiej będziemy mówić krótko o fixture.

🟢 Poziom A — rozumiem kontrakt i tworzę poprawny CSV

- Zbudujcie program w konfiguracji Release i wygenerujcie `raw_output.txt`.
- Opiszcie protokół, wersję, nagłówek i znaczenie każdego pola.
- Uruchomcie parser i zachowajcie jego `stdout`, `stderr` oraz kod zakończenia.
- Potwierdźcie, że powstało dokładnie `18` rekordów danych.
- Sprawdźcie CSV w edytorze tekstowym oraz przez import w Excelu.
- Wyjaśnijcie różnicę między parsowaniem a walidacją na konkretnym rekordzie.
- Pokażcie, że wartości `time_us` i `comparisons` zostały zapisane jako liczby bez jednostek w komórkach.

📦 *Do oddania — poziom A*

Przekażcie poniższy zestaw plików. W miejscu Kryptonim wpiszcie Wasz przydzielony kryptonim TiDPI.

```
Kryptonim\
└── TiDPI_06\
    ├── raport.md
    ├── main.cpp
    ├── parse_results.py
    ├── raw_output.txt
    ├── results.csv
    ├── logs\
    │   ├── parser_stdout.txt
    │   ├── parser_stderr.txt
    │   └── parser_exit_code.txt
    └── images\  (jeśli używacie obrazów)
```

Dołączcie źródło producenta C++, parser i rzeczywiste 18 rekordów w pliku surowym oraz wynikowym CSV. W raporcie opiszcie kontrakt i kontrolę importu w Excelu oraz edytorze; dowód importu może być opisem albo obrazem w images\. Osobny skoroszyt XLSX nie jest wymagany przez to zadanie. Oddajecie jedno sprawozdanie z oznaczeniem faktycznie wykonanego zakresu. Nazwy dodatkowych plików pokazane w drzewie są wzorcem organizacyjnym; jeżeli Wasz program używa innych nazw, zachowajcie działające odwołania i wskażcie te pliki w raporcie. Zapis <...> oznacza nazwę do zastąpienia, a nie dosłowną nazwę pliku. Wszystkie obrazy użyte w raporcie umieśćcie w images\ i odwołujcie się do nich względnie, np. ![Dowód D01](images/D01.png). Jeżeli nie używacie obrazów, katalog images\ nie jest wymagany. Pozostałe załączniki również linkujcie względnie.

🛡️ Poziom B — dowodzę, że parser odrzuca złe dane

- Sprawdźcie kolejno `invalid-number`, `missing-field`, `duplicate-row` i `wrong-version`.
- Dla każdego przypadku zapiszcie oczekiwaną oraz rzeczywistą decyzję parsera.
- Powiążcie błąd z numerem wiersza i konkretną regułą kontraktu.
- Przed jednym błędnym przebiegiem pozostawcie poprawny CSV z poprzedniej próby. Wykażcie, że parser usuwa stary plik i nie tworzy nowego.
- Sprawdźcie `failed-test` i wyjaśnijcie, dlaczego CSV powstaje mimo kodu `1`.
- Przygotujcie tabelę rozróżniającą: błąd odczytu, błąd struktury, błąd wartości oraz poprawne dane opisujące nieudany test.

Ukończenie wyłącznie poziomów A i B pozwala uzyskać najwyżej ocenę 4.

📁 *Do oddania — poziom B*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_06\
    ├── raport.md  (uzupełniony o A–B)
    └── przypadki\
        ├── invalid-number\
        ├── missing-field\
        ├── duplicate-row\
        ├── wrong-version\
        └── failed-test\
```

W każdym z pięciu katalogów zachowajcie wejście raw_output.txt, parser_stdout.txt, parser_stderr.txt i parser_exit_code.txt. W failed-test\ dołączcie również utworzony results.csv; w czterech odrzuconych przypadkach CSV ma nie istnieć. Oczekiwane i rzeczywiste decyzje, numery wierszy, reguły oraz dowód usunięcia starego CSV zapiszcie w raporcie. Nie oddawajcie starego CSV jako nowego wyniku.

🔧 Poziom C — rozszerzam kontrakt i walidację

Wykonajcie dwa z trzech badań:

- **Nowe pole `seed`.** Zaprojektujcie wersję `2` kontraktu, dodajcie seed do programu i parsera oraz wykażcie, że parser wersji `1` odrzuca nowe dane zamiast je błędnie interpretować.
- **Kontrola kompletności.** Dodajcie konfigurację oczekiwanych przypadków, rozmiarów i liczby prób. Parser ma wykryć brakujący rekord oraz nieoczekiwany rekord nadmiarowy.
- **Dane zawierające separator.** Dodajcie tekstowe pole `note`, którego poprawna wartość zawiera średnik i cudzysłów. Zmieńcie producenta tak, aby prawidłowo kodował CSV, i pokażcie, dlaczego ręczne sklejanie wiersza jest zawodne.

Poprawne ukończenie poziomów A–C otwiera drogę do oceny 5.

📦 *Do oddania — poziom C*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_06\
    ├── raport.md  (uzupełniony o A–C)
    └── rozszerzenia\
        └── C\
            ├── <badanie_1>\  (producent, parser, wejścia i wyniki)
            └── <badanie_2>\  (producent, parser, wejścia i wyniki)
```

Dołączcie źródła i testy pozytywne oraz negatywne dwóch wybranych badań. Dla seed zachowajcie wersję 1 i 2 parsera oraz dane wersji 2; dla kompletności — konfigurację oczekiwanych rekordów i wejścia z brakiem oraz nadmiarem; dla note — dane ze średnikiem i cudzysłowem. Dla każdego testu dołączcie strumienie i kod zakończenia, a CSV tylko wtedy, gdy został prawidłowo utworzony. Kontrakt i powiązanie plików opiszcie w raporcie.

🚀 Poziom D — testuję parser jak osobny produkt

Zaprojektujcie własny zestaw co najmniej dziesięciu fixture obejmujących przypadki graniczne i kombinacje błędów. Możliwe kierunki rozszerzenia:

- raport walidacji w osobnym, ustrukturyzowanym pliku JSON;
- bezpieczna obsługa wielu wersji kontraktu przez osobne funkcje parserów;
- wykrywanie nieoczekiwanych pustych wierszy, bardzo dużych liczb i powtórzonego nagłówka w środku pliku;
- test właściwości: każdy CSV zapisany przez producenta daje się ponownie odczytać bez zmiany wartości;
- inny problem uzgodniony z prowadzącym.

Ocena 6 wymaga samodzielnego modelu zagrożeń dla danych, uzasadnionych reguł, automatycznego uruchomienia fixture, dowodów zarówno akceptacji, jak i odrzucenia oraz obrony decyzji projektowych. Dziesięć kopii tego samego brakującego średnika nadal jest jednym pomysłem, tylko bardzo konsekwentnym.

📦 *Do oddania — poziom D*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_06\
    ├── raport.md  (uzupełniony o A–D)
    └── rozszerzenia\
        └── D\
            ├── parse_results.py  (badana wersja)
            ├── run_fixtures.py
            ├── fixtures\  (co najmniej 10 plików wejściowych)
            └── wyniki\  (decyzje, strumienie i kody każdego testu)
```

Dołączcie skrypt automatycznie uruchamiający Wasze fixture i zestaw oczekiwanych oraz rzeczywistych decyzji, z jednoznacznym powiązaniem do wejść. Jeśli wybraliście raport walidacji JSON, zachowajcie także te raporty. W sprawozdaniu opiszcie model zagrożeń, reguły, dowody akceptacji i odrzucenia. Nie twórzcie wynikowego CSV dla odrzuconego wejścia.

📝 Sprawozdanie

Oddajecie jedno indywidualne sprawozdanie w Markdownie zgodne z zasadami lekcji 01. Raport może zostać przekazany internetowemu agentowi AI do wstępnej analizy, dlatego musi być całkowicie zanonimizowany.

🔒 Obowiązkowa anonimizacja

- użyjcie wyłącznie przydzielonego kryptonimu TiDPI;
- nie umieszczajcie imienia, nazwiska, adresu e-mail ani nazwy konta w treści, nazwie pliku, metadanych, ścieżkach, zrzutach i innych dowodach;
- sprawdźcie surowe komunikaty, właściwości dokumentów DOCX lub PDF oraz kadrowanie zrzutów;
- mapę `kryptonim TiDPI ↔ uczeń` przechowuje wyłącznie nauczyciel.

🧱 Wymagana struktura raportu

- `Kryptonim TiDPI i wykonany poziom;`
- `Cel oraz kryteria akceptacji danych;`
- `Metryczka środowiska i wersji programów;`
- `Kontrakt i schemat danych;`
- `Procedura generowania, parsowania i walidacji;`
- `Przypadek poprawny;`
- `Przypadki odrzucone;`
- `Poprawne dane opisujące nieudany test;`
- `Kontrola braku starego CSV;`
- `Rejestr dowodów D01, D02…;`
- `Rozszerzenia poziomu C lub D;`
- `Ograniczenia parsera;`
- `Wnioski;`
- `Wykorzystanie AI i sposób weryfikacji.`

Każdą decyzję parsera powiążcie z konkretnym plikiem, kodem zakończenia i regułą. Nie wystarczy napisać „parser działa”, jeżeli raport pokazuje wyłącznie przypadek poprawny.

Użycie AI jest dozwolone. Wskażcie zgodnie z prawdą narzędzie, zakres pomocy i sposób sprawdzenia kodu oraz wniosków. Musicie potrafić wyjaśnić każdą regułę walidacji i przewidzieć jej zachowanie dla nowego przykładu.

📊 Progi i kryteria oceny

Wyższy próg obejmuje wymagania wszystkich niższych progów. Tabela wskazuje maksymalną możliwą ocenę za poprawnie wykonany, udokumentowany i terminowo oddany zakres.

Maksymalna ocena	Wymagany zakres	Minimalne dowody
2	częściowe wykonanie poziomu A	surowy output i próba uruchomienia parsera
3	poprawnie ukończony poziom A	kontrakt, 18 zaakceptowanych rekordów, kod `0`, CSV i kontrola importu
4	poziomy A i B	cztery odrzucone formaty, `failed-test`, kody `0/1/2`, komunikaty z numerami wierszy i brak starego CSV

Maksymalna ocena	Wymagany zakres	Minimalne dowody
5	poziomy A–C	dwa spójne rozszerzenia producenta, kontraktu i parsera oraz ich testy pozytywne i negatywne
6	poziomy A–C oraz wartościowe badanie D	szeroki zestaw fixture, wiedza ponadprogramowa, uzasadnione reguły, automatyzacja i obrona pracy

Niezależnie od poziomu oceniane są:

- jednoznaczność kontraktu i jednostek;
- rozdzielenie parsowania od walidacji;
- kontrola wersji, nagłówka, liczby pól, typów, zakresów i duplikatów;
- rozróżnienie błędnych danych od poprawnego rekordu nieudanego testu;
- brak częściowego albo starego CSV po odrzuceniu wejścia;
- zachowanie surowych dowodów i kodów zakończenia;
- zgodność raportu z rzeczywistymi plikami;
- anonimizacja, czytelność i oznaczenie dowodów;
- rozumienie oraz umiejętność obrony pracy.

Agent AI wspiera nauczyciela w odnajdywaniu dowodów i porównywaniu raportu z jawnymi kryteriami. Nie wystawia samodzielnie oceny. Materiał przygotowany z pomocą AI, którego nie potraficie zweryfikować albo obronić, nie jest dowodem Waszych umiejętności i może drastycznie obniżyć ocenę, aż do oceny niedostatecznej.

🚧 Typowe pułapki

- projektowanie komunikatu dla człowieka, a następnie ratowanie go coraz dłuższym regexem;
- ciche pomijanie nierozpoznanych wierszy;
- sprawdzanie tylko liczby kolumn bez kontroli typów i zakresów;
- zaakceptowanie `True`, `TRUE`, `yes` i `1` bez jawnej decyzji kontraktu;
- pomylenie `correct=false` z uszkodzonym rekordem;
- zapisanie części poprawnych rekordów mimo błędu w innym miejscu pliku;
- pozostawienie starego CSV po nieudanej walidacji;
- ręczne sklejanie CSV bez obsługi separatorów i cudzysłowów;
- brak `newline=''` przy pracy z modułem `csv` w Windows;
- wpisanie jednostki do wartości i ponowne dopisywanie jej w nagłówku;
- uznanie samego otwarcia pliku w Excelu za pełną walidację;
- przedstawienie małych czasów z tego ćwiczenia jako rzetelnego benchmarku;
- poprawianie surowego dowodu zamiast utworzenia jawnej fixture;
- bezrefleksyjne przyjęcie parsera wygenerowanego przez agenta AI.

🏁 Co powinniście zapamiętać?

- Ustrukturyzowany output jest kontraktem pomiędzy producentem a odbiorcą danych.
- Wersja formatu chroni parser przed cichą, błędną interpretacją zmian.
- Schemat określa pola, typy, jednostki, zakresy i dozwolone wartości.
- Parsowanie buduje strukturę; walidacja ocenia jej zgodność z regułami.
- Poprawny rekord może informować o nieudanym teście.
- Parser nie powinien cicho pomijać błędnych wierszy ani zapisywać częściowego wyniku.
- Stary CSV po odrzuconym wejściu jest niebezpiecznym, mylącym artefaktem.
- Moduł `csv` jest bezpieczniejszy od ręcznego sklejania wierszy.
- Regex jest użyteczny dla tekstu zastanego, ale nie zastępuje typów, zakresów i reguł biznesowych.
- Wiarygodność parsera pokazują przede wszystkim dobrze dobrane przypadki błędne, nie tylko jeden szczęśliwy przebieg.